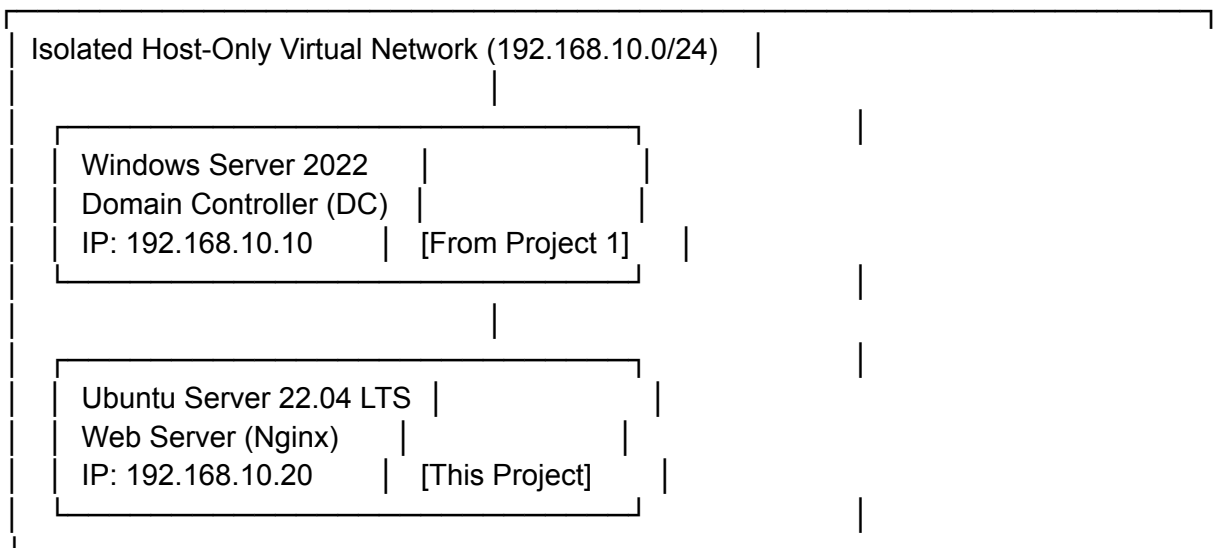


Project 2: Hardened Linux Server, Backup Automation & Disaster Recovery

◆ Objective & Network Topology

The goal of this project was to provision, harden, and automate a production-grade Ubuntu Server 22.04 LTS instance integrated into the existing Active Directory lab environment. The server simulates a corporate internal web service, hardened following CIS benchmark controls, with a fully automated backup and verified disaster recovery workflow.



Environment Specs:

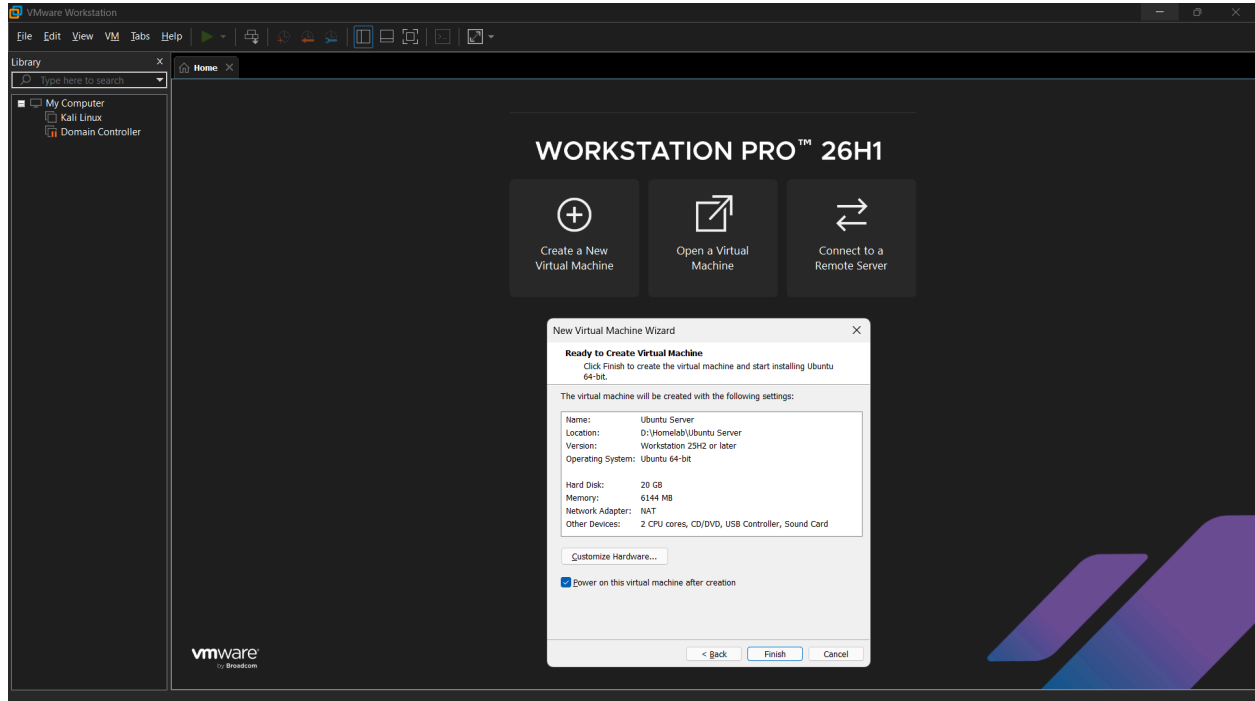
- **Operating System:** Ubuntu Server 22.04 LTS (Minimal Install, No GUI)
- **Hostname:** ubuntu-srv
- **Static IP:** 192.168.10.20
- **Network:** Same Host-Only segment as Domain Controller
- **Role:** Nginx Web Server, Backup Host, future Wazuh Agent

◆ Step-by-Step Implementation

1. VM Provisioning & Static IP Configuration

Before deploying any services, the Ubuntu Server was provisioned as a minimal install — no GUI, no unnecessary packages — mirroring real-world production Linux server standards.

- Created a new VMware Workstation Pro VM (2GB RAM, 2 cores, 20GB disk)
- Selected Ubuntu Server 22.04 LTS ISO with **Minimal Install** option
- Enabled OpenSSH Server during installation setup
- Assigned static IP during installation to prevent DNS resolution issues across the lab subnet



Network configuration

[Help]

Configure at least one interface this server can use to talk to other machines, and which preferably provides sufficient access for updates.

NAME	TYPE	NOTES
[ens33	eth	-
▶]		
DHCPv4	192.168.57.131/24	
00:0c:29:f4:f8:4c / Intel Corporation / 82545EM Gigabit Ethernet Controller (Copper) (PRO/1000 MT Single Port Adapter)		

[Create bond ▶]

Edit ens33 IPv4 configuration

IPv4 Method: [Manual ▼]

Subnet: 192.168.10.0/24

Address: 192.168.10.20

Gateway: 192.168.10.1

Name servers: 192.168.10.10, 8.8.8.8
IP addresses, comma separated

Search domains: lab.corp.local
Domains, comma separated

[Save]

[Cancel]

[Done]

[Back]

Installation complete!

[Help]

```
curtin command install
writing install sources to disk
running 'curtin extract'
curtin command extract
acquiring and extracting image from cp:///tmp/tmp66h67wb0/mount
configuring keyboard
curtin command in-target
executing curtin install curthooks step
curtin command install
configuring installed system
running 'curtin curthooks'
curtin command curthooks
configuring apt configuring apt
installing missing packages
Installing packages on target system: ['grub-pc']
configuring iscsi service
configuring raid (mdadm) service
configuring NVMe over TCP
installing kernel
setting up swap
apply networking config
writing etc/fstab
configuring multipath
updating packages on target system
configuring pollinate user-agent on target
updating inittab configuration
configuring target system bootloader
installing grub to target devices
copying metadata from /cdrom
final system configuration
calculating extra packages to install
installing openssh-server
retrieving openssh-server
curtin command system-install
unpacking openssh-server
curtin command system-install
configuring cloud-init
restoring apt configuration
subiquityLate/run:
```

[View full log]

[Reboot Now]

```
[ 38.273988] cloud-init[1041]: Cloud-init v. 24.2-0ubuntu1~22.04.1 finished at Sat, 13 Jun 2026 09:07:11 +0000. Datasource DataSourceNone. Up 38.26 seconds
```

```
Login incorrect
ubuntu_server login: ubuntu_server
Password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-119-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro
```

System information as of Sat Jun 13 09:07:22 AM UTC 2026

```
System load:  0.64      Processes:            239
Usage of /:   30.2% of 9.75GB   Users logged in:     0
Memory usage: 6%      IPv4 address for ens33: 192.168.10.20
Swap usage:   0%
```

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

Failed to connect to <https://changelogs.ubuntu.com/meta-release-lts>. Check your Internet connection or proxy settings

Last login: Sat Jun 13 09:05:45 UTC 2026 on tty1
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu_server@ubuntu_server:~\$

```
ubuntu_server@ubuntu_server:~$ ping -c 3 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=128 time=1.34 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=128 time=1.82 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=128 time=1.74 ms

--- 192.168.10.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 1.342/1.632/1.815/0.207 ms
ubuntu_server@ubuntu_server:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=52.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=52.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=52.7 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 52.069/52.573/52.944/0.369 ms
ubuntu_server@ubuntu_server:~$
```

```
C:\Users\Administrator>ping 192.168.10.20

Pinging 192.168.10.20 with 32 bytes of data:
Reply from 192.168.10.20: bytes=32 time<1ms TTL=64
Reply from 192.168.10.20: bytes=32 time=1ms TTL=64
Reply from 192.168.10.20: bytes=32 time=1ms TTL=64
Reply from 192.168.10.20: bytes=32 time<1ms TTL=64
```

2. Nginx Web Server Deployment

Nginx was deployed as a mock corporate internal web portal — simulating a real service running inside the corporate network.

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install nginx -y
```

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

```
ubuntu_server@ubuntu-server:~$ sudo systemctl status nginx
[sudo] password for ubuntu_server:
• nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2026-06-17 08:16:27 UTC; 1min 22s ago
     Docs: man:nginx(8)
  Process: 4767 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 4768 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 4863 (nginx)
   Tasks: 3 (limit: 6925)
  Memory: 4.6M
    CPU: 25ms
   CGroup: /system.slice/nginx.service
           └─4863 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─4865 "nginx: worker process"
             └─4866 "nginx: worker process"

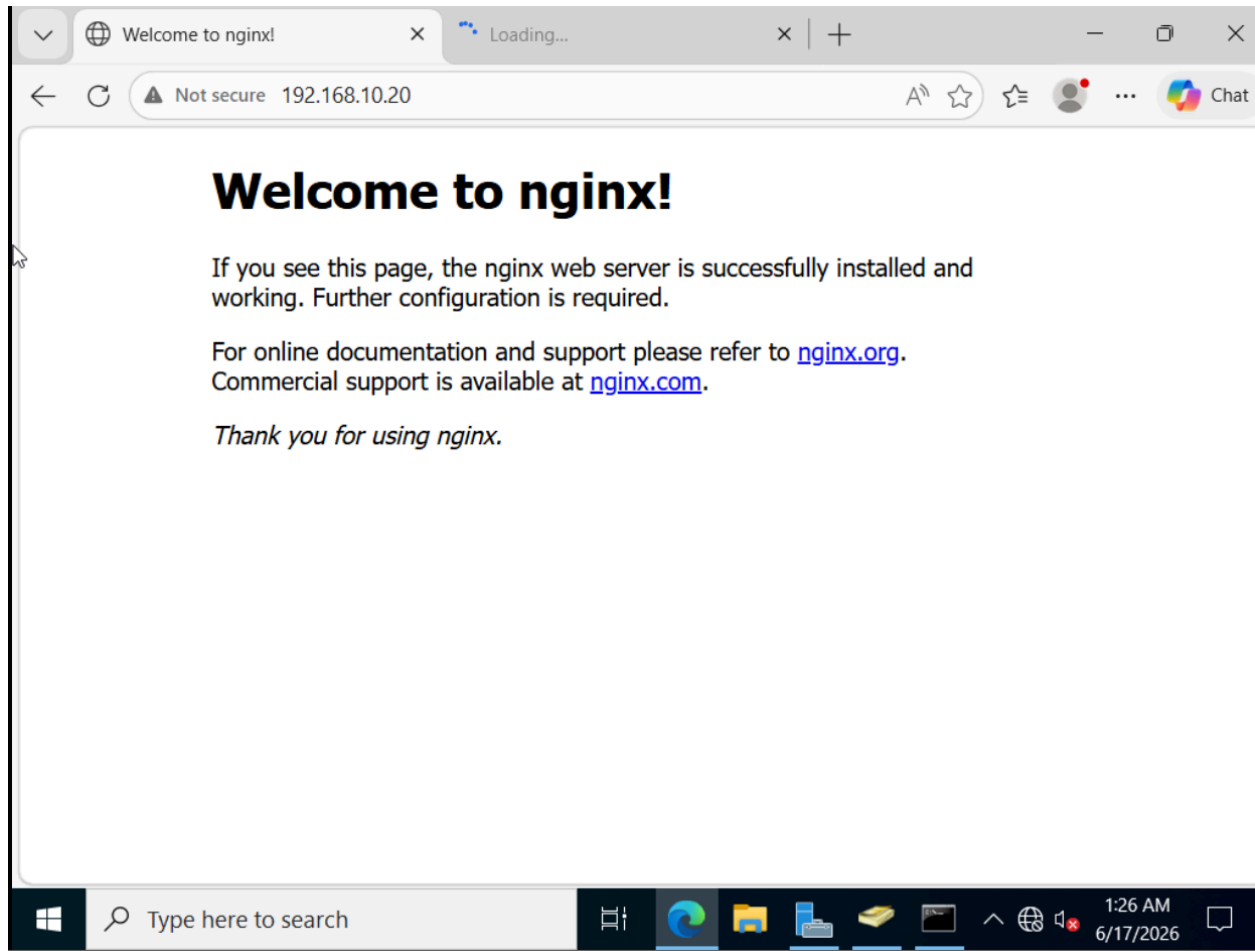
Jun 17 08:16:27 ubuntu-server systemd[1]: Starting A high performance web server and a reverse proxy: nginx.service.
Jun 17 08:16:27 ubuntu-server systemd[1]: Started A high performance web server and a reverse proxy: nginx.service.
lines 1-17/17 (END)
```

```
ubuntu_server@ubuntu-server:~$ sudo systemctl start nginx
ubuntu_server@ubuntu-server:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executed: /lib/systemd/systemd-sysv-install enable nginx
ubuntu_server@ubuntu-server:~$
```

```
ubuntu_server@ubuntuserver:~$ curl http://192.168.10.20
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu_server@ubuntuserver:~$
```



[Screenshot Placeholder: nginx-status.png] Caption: Nginx service active and enabled on boot

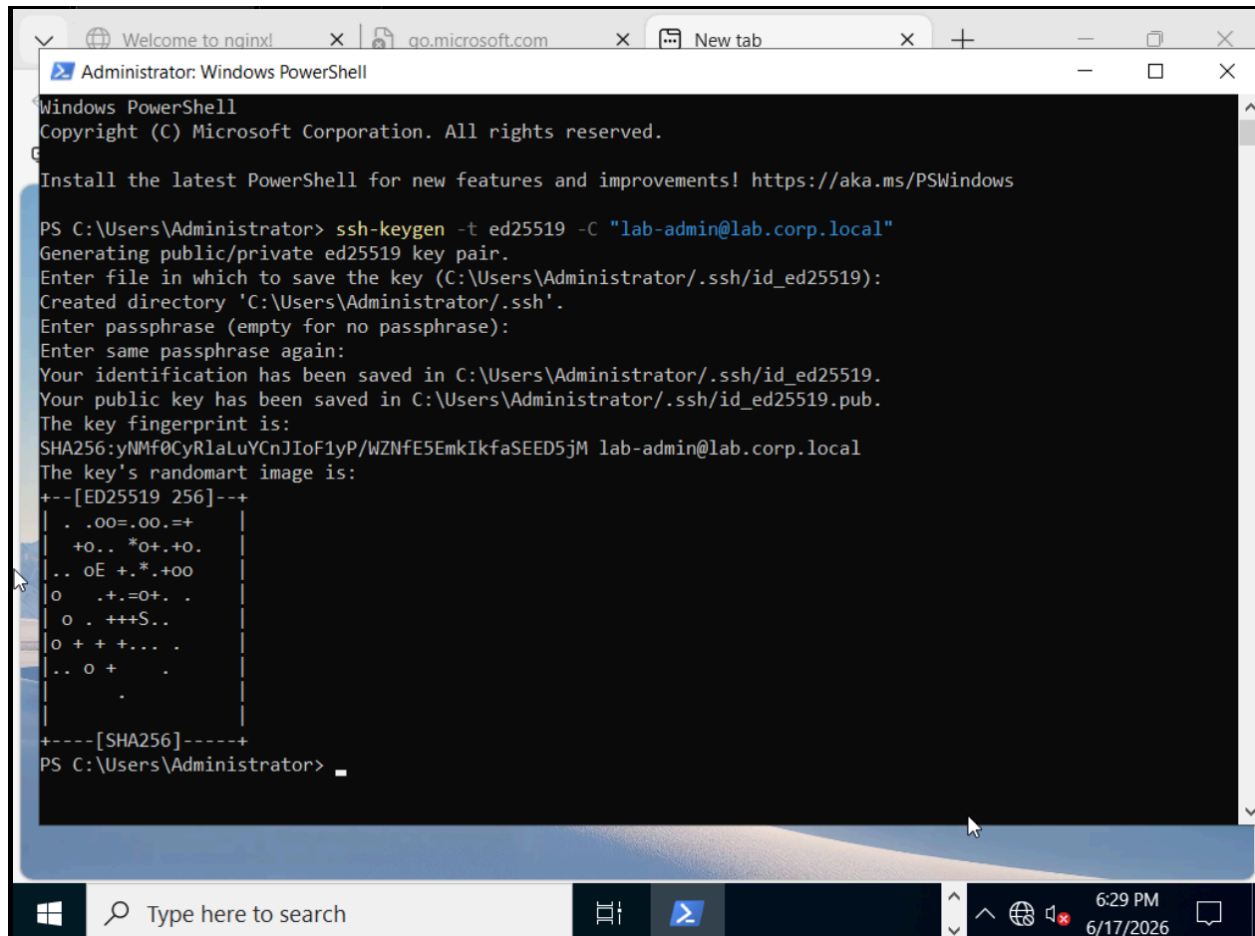
[Screenshot Placeholder: nginx-browser.png] Caption: Custom internal portal page accessible from Windows VM at <http://192.168.10.20>

3. SSH Hardening

The default SSH configuration on Ubuntu Server accepts password-based logins on port 22 — the most commonly brute-forced attack vector on Linux systems. This was completely locked down.

Step 1: Generate SSH Key Pair (from Windows VM)

```
ssh-keygen -t ed25519 -C "lab-admin@lab.corp.local"
```



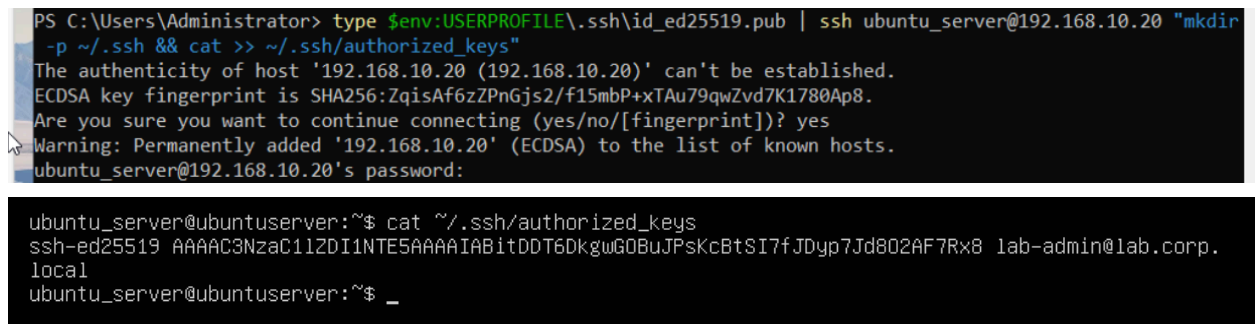
```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> ssh-keygen -t ed25519 -C "lab-admin@lab.corp.local"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\Administrator\.ssh/id_ed25519):
Created directory 'C:\Users\Administrator\.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\Administrator\.ssh/id_ed25519.
Your public key has been saved in C:\Users\Administrator\.ssh/id_ed25519.pub.
The key fingerprint is:
SHA256:yNMf0CyRlaLuYCnJIof1yP/WZNfE5EmkIkfaSEED5jM lab-admin@lab.corp.local
The key's randomart image is:
+--[ED25519 256]--+
| .oo=.oo.=+ |
| +o.. *o+.+o. |
| .. oE +.*.+oo |
| o .+. =o+. . |
| o . +++S.. |
| o + + +... . |
| .. o + . |
| . |
+----[SHA256]-----+
PS C:\Users\Administrator>
```

Step 2: Copy Public Key to Ubuntu Server

type %env:USERPROFILE\.ssh\id_ed25519.pub | ssh ubuntu_server@192.168.10.20 "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"



```
PS C:\Users\Administrator> type %env:USERPROFILE\.ssh\id_ed25519.pub | ssh ubuntu_server@192.168.10.20 "mkdir
-p ~/.ssh && cat >> ~/.ssh/authorized_keys"
The authenticity of host '192.168.10.20 (192.168.10.20)' can't be established.
ECDSA key fingerprint is SHA256:ZqisAf6zZPnGjs2/f15mbP+xTAu79qwZvd7K1780Ap8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.10.20' (ECDSA) to the list of known hosts.
ubuntu_server@192.168.10.20's password:

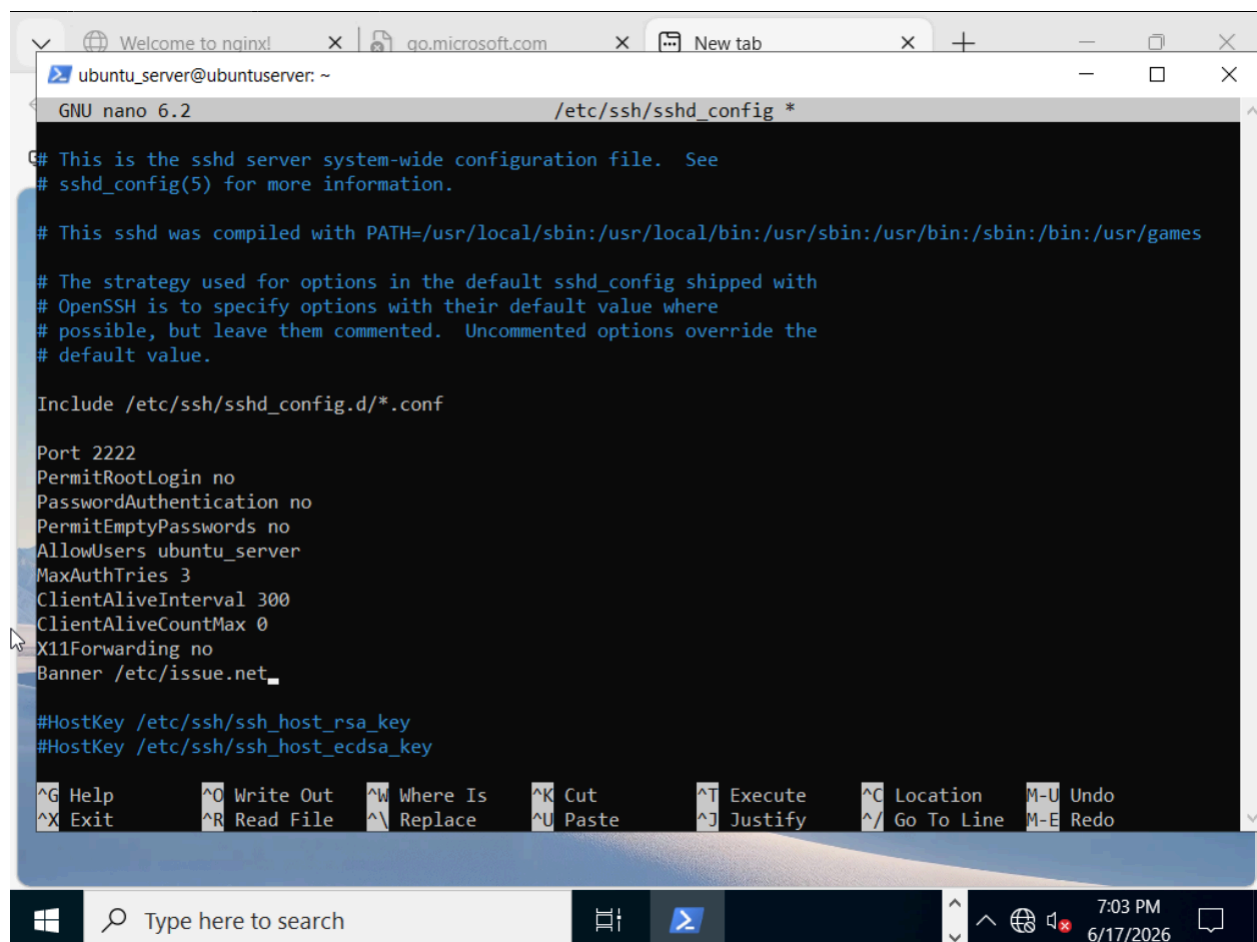
ubuntu_server@ubuntuserver:~$ cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIABitDDT6DkgwG0BuJPskcBtSI7fJDyp7Jd802AF7Rx8 lab-admin@lab.corp.
local
ubuntu_server@ubuntuserver:~$
```

Step 3: Harden SSH Daemon Configuration

/etc/ssh/sshd_config — changes applied

Port 2222
PermitRootLogin no
PasswordAuthentication no
PermitEmptyPasswords no
AllowUsers charles
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 0
X11Forwarding no
Banner /etc/issue.net

sudo systemctl restart sshd



```
GNU nano 6.2 /etc/ssh/sshd_config *
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2222
PermitRootLogin no
PasswordAuthentication no
PermitEmptyPasswords no
AllowUsers ubuntu_server
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 0
X11Forwarding no
Banner /etc/issue.net

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> ssh -i $env:USERPROFILE\.ssh\id_ed25519 -p 2222 ubuntu_server@192.168.10.20
Ubuntu 22.04.5 LTS
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-119-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Jun 18 02:07:38 AM UTC 2026

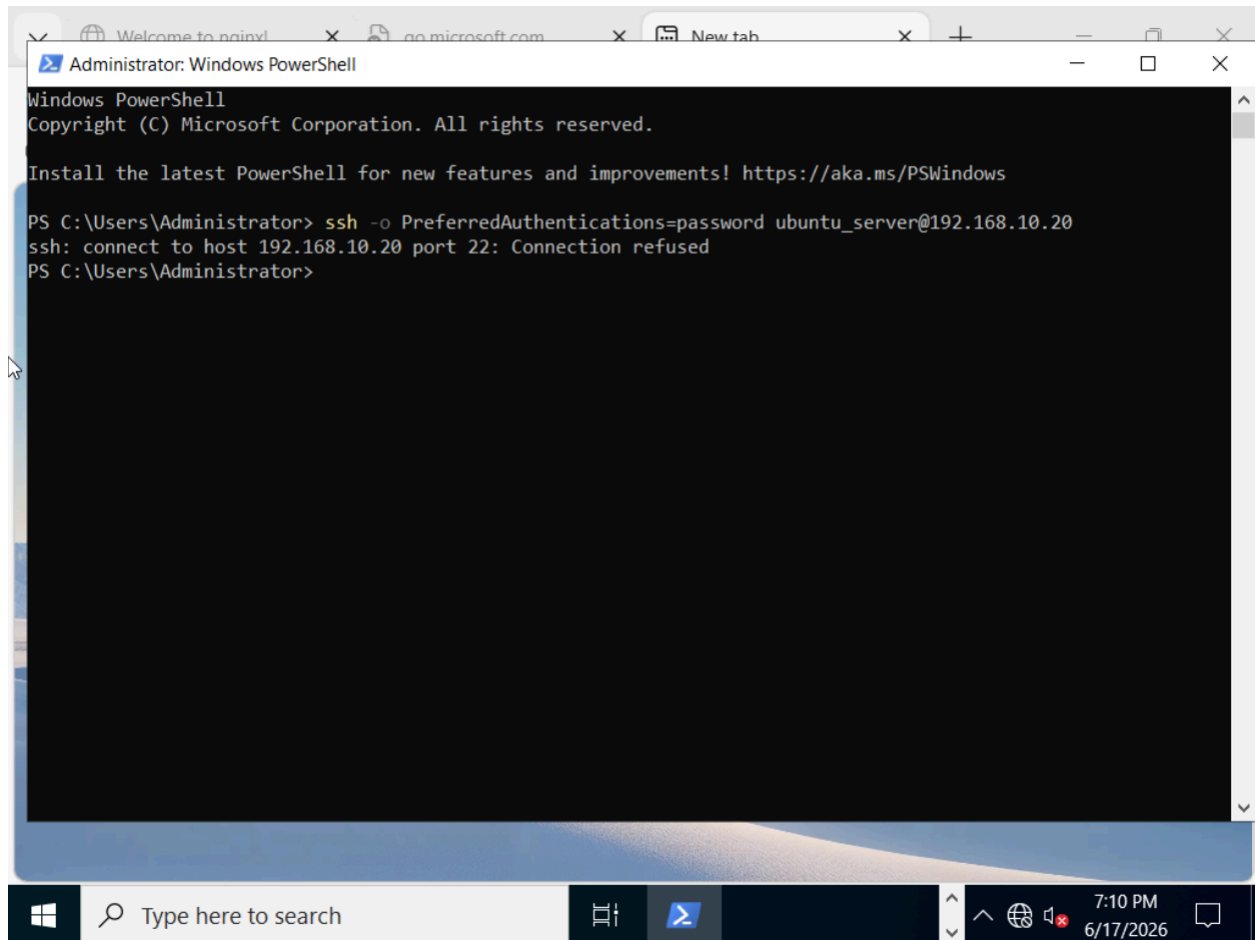
System load: 0.0          Processes:            220
Usage of /:  37.6% of 9.75GB Users logged in:        1
Memory usage: 8%          IPv4 address for ens33: 192.168.57.131
Swap usage:  0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

240 updates can be applied immediately.
180 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
```



[Screenshot Placeholder: sshd-config.png] Caption: Hardened SSH daemon configuration — password auth disabled, non-standard port

[Screenshot Placeholder: ssh-key-login.png] Caption: Successful key-based SSH login from Windows VM on port 2222

[Screenshot Placeholder: ssh-password-rejected.png] Caption: Password authentication rejected — only key-based login accepted

4. UFW Firewall Configuration

A default-deny firewall posture was applied — all inbound traffic is dropped unless explicitly permitted from the lab subnet.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

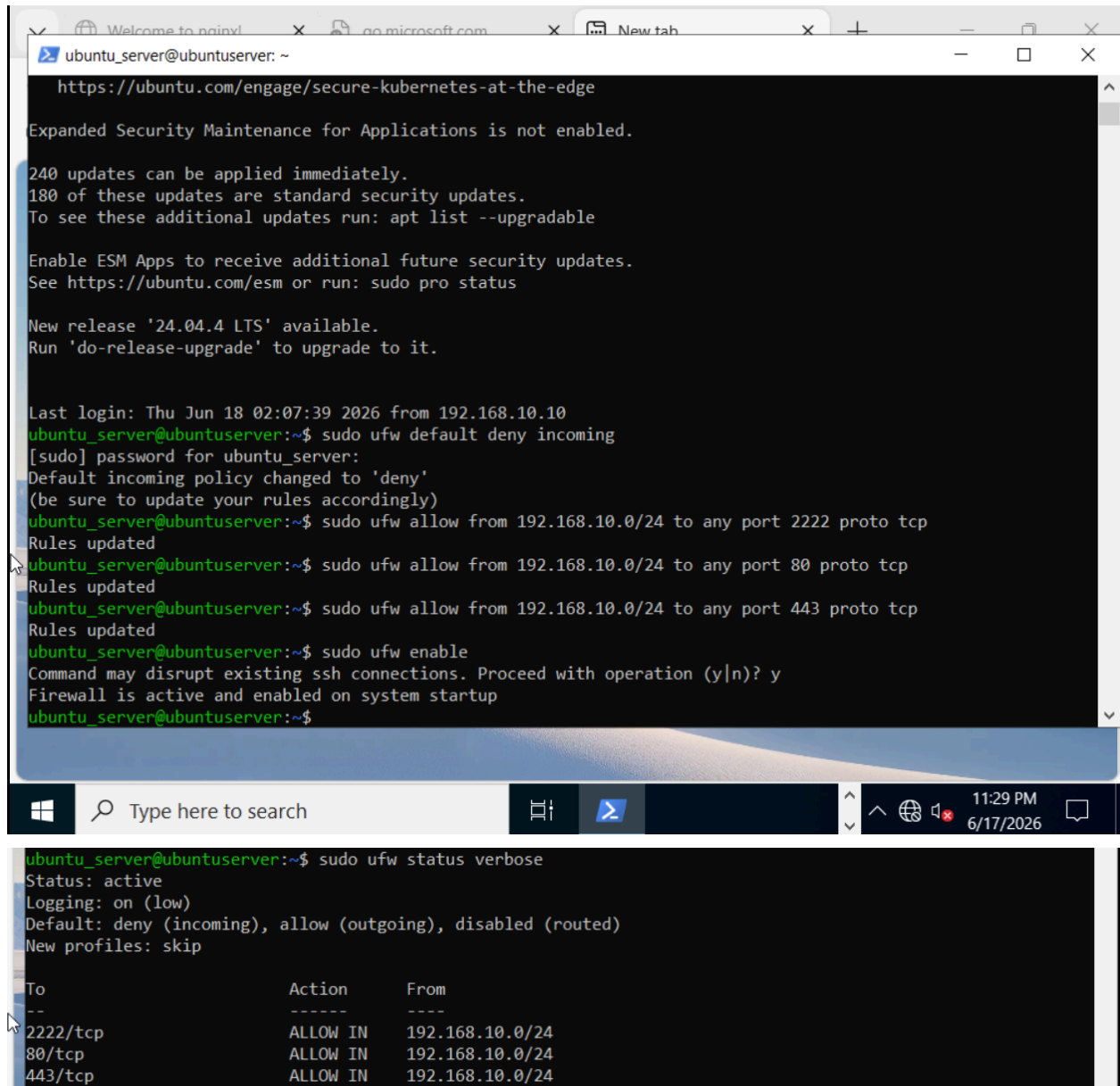
Allow only from lab subnet

sudo ufw allow from 192.168.10.0/24 to any port 2222 proto tcp

sudo ufw allow from 192.168.10.0/24 to any port 80 proto tcp

sudo ufw allow from 192.168.10.0/24 to any port 443 proto tcp

sudo ufw enable



The screenshot shows a terminal window on an Ubuntu server. The user has configured UFW with a default deny policy and three specific allow rules for ports 2222, 80, and 443 from the 192.168.10.0/24 subnet. The firewall is now active. The terminal output is as follows:

```
ubuntu_server@ubuntuserver: ~  
https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
Expanded Security Maintenance for Applications is not enabled.  
240 updates can be applied immediately.  
180 of these updates are standard security updates.  
To see these additional updates run: apt list --upgradable  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
New release '24.04.4 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
Last login: Thu Jun 18 02:07:39 2026 from 192.168.10.10  
ubuntu_server@ubuntuserver:~$ sudo ufw default deny incoming  
[sudo] password for ubuntu_server:  
Default incoming policy changed to 'deny'  
(be sure to update your rules accordingly)  
ubuntu_server@ubuntuserver:~$ sudo ufw allow from 192.168.10.0/24 to any port 2222 proto tcp  
Rules updated  
ubuntu_server@ubuntuserver:~$ sudo ufw allow from 192.168.10.0/24 to any port 80 proto tcp  
Rules updated  
ubuntu_server@ubuntuserver:~$ sudo ufw allow from 192.168.10.0/24 to any port 443 proto tcp  
Rules updated  
ubuntu_server@ubuntuserver:~$ sudo ufw enable  
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y  
Firewall is active and enabled on system startup  
ubuntu_server@ubuntuserver:~$  
ubuntu_server@ubuntuserver:~$ sudo ufw status verbose  
Status: active  
Logging: on (low)  
Default: deny (incoming), allow (outgoing), disabled (routed)  
New profiles: skip  


| To       | Action   | From            |
|----------|----------|-----------------|
| --       | -----    | ----            |
| 2222/tcp | ALLOW IN | 192.168.10.0/24 |
| 80/tcp   | ALLOW IN | 192.168.10.0/24 |
| 443/tcp  | ALLOW IN | 192.168.10.0/24 |

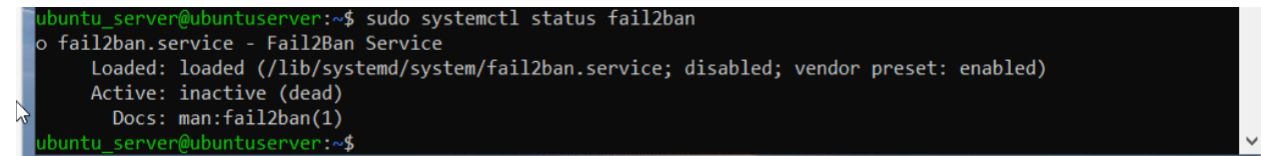

```

[Screenshot Placeholder: ufw-status.png] Caption: UFW active with default-deny posture and explicit lab-subnet-only allow rules

5. Fail2Ban — Brute Force Protection

Even with password authentication disabled, fail2ban was deployed as an additional detection and response layer — automatically banning IPs that show suspicious login behavior.

```
sudo apt install fail2ban -y
```

A terminal window with a dark background and green text. The prompt is 'ubuntu_server@ubuntu-server:~\$'. The command 'sudo systemctl status fail2ban' has been executed. The output shows the status of the 'fail2ban.service' as 'disabled' and 'inactive (dead)'.

```
ubuntu_server@ubuntu-server:~$ sudo systemctl status fail2ban
o fail2ban.service - Fail2Ban Service
   Loaded: loaded (/lib/systemd/system/fail2ban.service; disabled; vendor preset: enabled)
   Active: inactive (dead)
     Docs: man:fail2ban(1)
ubuntu_server@ubuntu-server:~$
```

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

```
sudo nano /etc/fail2ban/jail.local
```

jail.local — SSH jail configuration

[sshd]

enabled = true

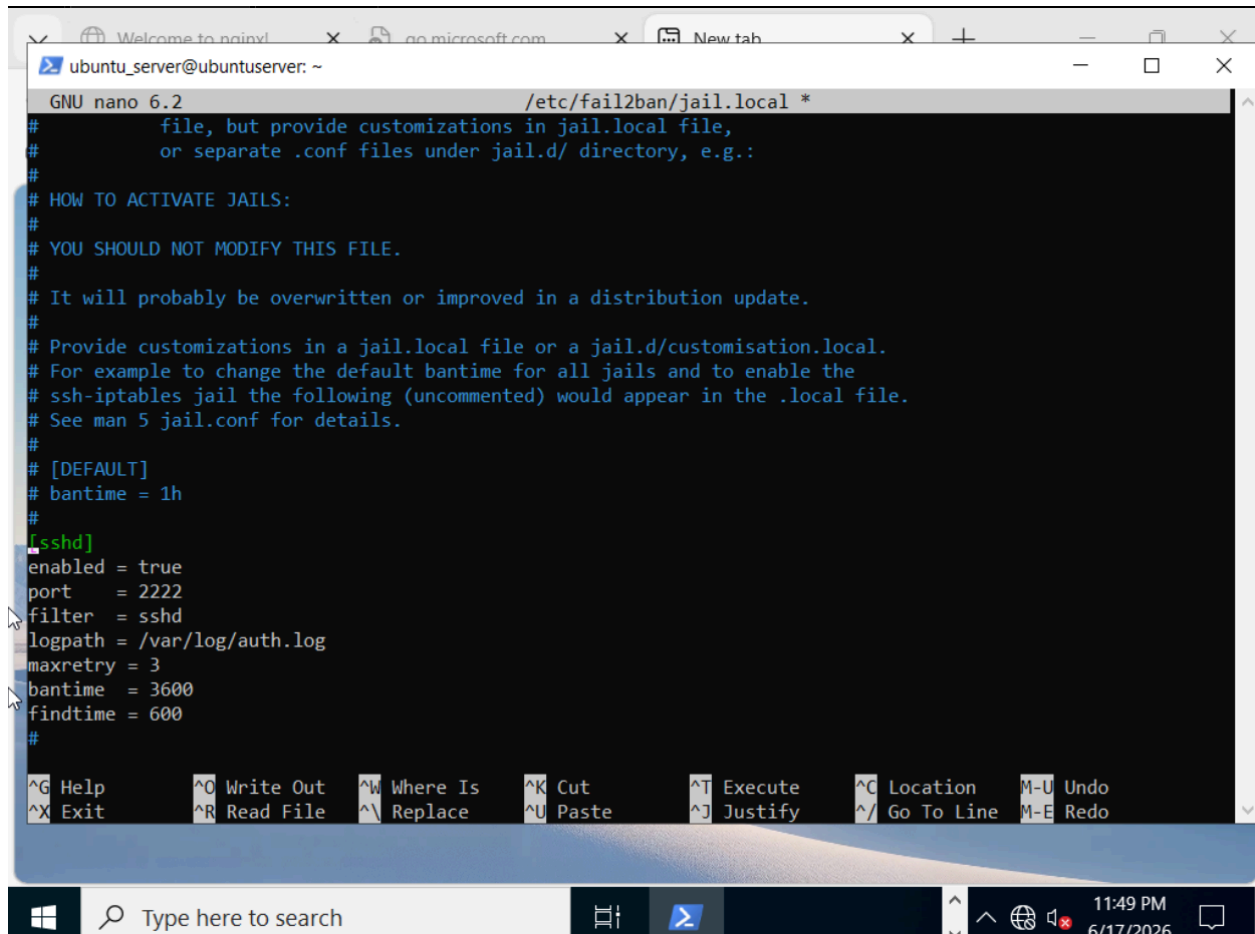
port = 2222

logpath = /var/log/auth.log

maxretry = 3

bantime = 3600

findtime = 600

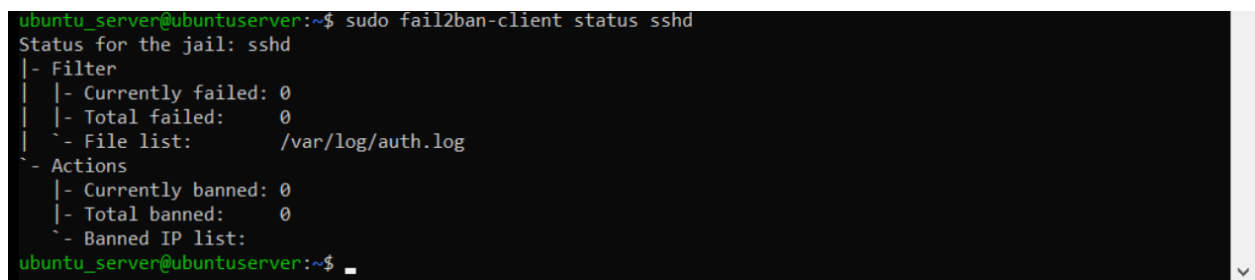


```
ubuntu_server@ubuntu: ~  
GNU nano 6.2 /etc/fail2ban/jail.local *  
# file, but provide customizations in jail.local file,  
# or separate .conf files under jail.d/ directory, e.g.:  
#  
# HOW TO ACTIVATE JAILS:  
#  
# YOU SHOULD NOT MODIFY THIS FILE.  
#  
# It will probably be overwritten or improved in a distribution update.  
#  
# Provide customizations in a jail.local file or a jail.d/customisation.local.  
# For example to change the default bantime for all jails and to enable the  
# ssh-iptables jail the following (uncommented) would appear in the .local file.  
# See man 5 jail.conf for details.  
#  
# [DEFAULT]  
# bantime = 1h  
#  
[sshd]  
enabled = true  
port = 2222  
filter = sshd  
logpath = /var/log/auth.log  
maxretry = 3  
bantime = 3600  
findtime = 600  
#  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

sudo systemctl enable --now fail2ban

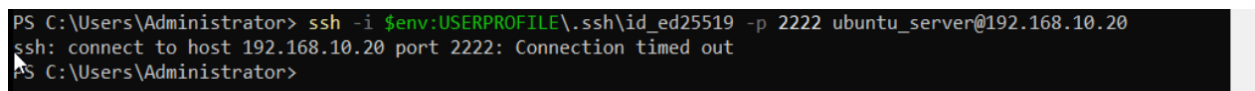
Verify

sudo fail2ban-client status sshd



```
ubuntu_server@ubuntu:~$ sudo fail2ban-client status sshd  
Status for the jail: sshd  
|- Filter  
| |- Currently failed: 0  
| |- Total failed: 0  
| `-- File list: /var/log/auth.log  
`- Actions  
   |- Currently banned: 0  
   |- Total banned: 0  
   `-- Banned IP list:  
ubuntu_server@ubuntu:~$
```

(After 3 failed tries)



```
PS C:\Users\Administrator> ssh -i %env:USERPROFILE\.ssh\id_ed25519 -p 2222 ubuntu_server@192.168.10.20  
ssh: connect to host 192.168.10.20 port 2222: Connection timed out  
PS C:\Users\Administrator>
```

```
Status for the jail: sshd
|- Filter
| | - Currently failed: 0
| | - Total failed: 3
| | - File list: /var/log/auth.log
|- Actions
| | - Currently banned: 1
| | - Total banned: 1
| | - Banned IP list: 192.168.10.10
ubuntu_server@ubuntuserver:~$ _
```

6. CIS Benchmark Hardening

System-level hardening was applied following the CIS Ubuntu 22.04 Benchmark — the same standard used by enterprise security teams and compliance auditors.

1. Package Cleanup

```
sudo apt purge -y telnet rsh-client talk inetutils-telnetd xinetd
```

```
sudo apt autoremove -y
```

```
sudo apt autoclean
```

2. Kernel Hardening (sysctl)

```
sudo nano /etc/sysctl.d/99-hardening.conf
```

```
net.ipv4.ip_forward = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.default.accept_source_route = 0
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.log_martians = 1
kernel.kptr_restrict = 2
kernel.dmesg_restrict = 1
kernel.sysrq = 0
kernel.yama.ptrace_scope = 1
```

```
sudo sysctl -p /etc/sysctl.d/99-hardening.conf
```

3. Password Policy

```
sudo apt install libpam-pwquality -y
sudo nano /etc/security/pwquality.conf
```

```
minlen = 14
ucredit = -1
lcredit = -1
dcredit = -1
ocredit = -1
difok = 8
```

4. File Permissions

```
sudo chmod 640 /etc/shadow
sudo chmod 644 /etc/passwd
sudo chmod 700 /etc/cron.d /etc/cron.daily /etc/cron.hourly /etc/cron.weekly /etc/cron.monthly
```

5. Auto Security Updates

```
sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure -plow unattended-upgrades
```

Additional CIS Controls Applied:

- Removed unnecessary packages (telnet, rsh-client, talk)
- Disabled unused services (avahi-daemon, cups)
- Configured PAM password quality (minimum length 14, complexity requirements)
- Set password expiry policy (90-day max, 14-day warning)
- Hardened critical file permissions (/etc/shadow 640, /etc/passwd 644)
- Configured login warning banner (/etc/issue.net)
- Enabled automatic security updates (unattended-upgrades)

```
ubuntu_server@ubuntu-server:~$ sudo sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
ubuntu_server@ubuntu-server:~$ sudo sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 1
ubuntu_server@ubuntu-server:~$ ls -la /etc/shadow
-rw-r----- 1 root shadow 1063 Jun 13 09:05 /etc/shadow
ubuntu_server@ubuntu-server:~$
```

7. Backup Automation Script

A production-style backup script was written in Bash to handle compression, timestamping, logging, and automated retention management.

Make directory for the backup


```
sudo mkdir -p /backups/daily
sudo mkdir -p /backups/logs
sudo chmod 700 /backups /backups/daily /backups/logs
```

```
ubuntu_server@ubuntu-server: ~
ubuntu_server@ubuntu-server:~$ sudo mkdir -p /backups/daily
[sudo] password for ubuntu_server:
ubuntu_server@ubuntu-server:~$ sudo mkdir -p /backups/logs

ubuntu_server@ubuntu-server:~$ sudo chmod 700 /backups /backups/daily /backups/logs

ubuntu_server@ubuntu-server:~$ _
```

Then create the backup script:

sudo nano /usr/local/bin/[backup.sh](#)

```
ubuntu_server@ubuntu-server: ~
GNU nano 6.2 /usr/local/bin/backup.sh *
#!/bin/bash
BACKUP_DIR="/backups/daily"
LOG_FILE="/backups/logs/backup.log"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
HOSTNAME=$(hostname)
ARCHIVE_NAME="$HOSTNAME-backup-$DATE.tar.gz"
RETENTION_DAYS=7

SOURCES="/etc" "/var/www/html" "/var/log")

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

log "Starting backup: $ARCHIVE_NAME"
tar -czf "$BACKUP_DIR/$ARCHIVE_NAME" "${SOURCES[@]} 2>/dev/null

if [ $? -eq 0 ]; then
    SIZE=$(du -sh "$BACKUP_DIR/$ARCHIVE_NAME" | cut -f1)
    log "Backup successful: $ARCHIVE_NAME ($SIZE)"
else
    log "ERROR: Backup failed"
    exit 1
fi

find "$BACKUP_DIR" -name "*.tar.gz" -mtime +$RETENTION_DAYS -delete
log "Retention cleanup complete"
```

Change permissions and run it

sudo chmod 700 /usr/local/bin/[backup.sh](#)

sudo /usr/local/bin/[backup.sh](#)

```
ubuntu_server@ubuntu-server:~$ sudo /usr/local/bin/backup.sh
[2026-06-19 10:12:14] Starting backup: ubuntu-server-backup-2026-06-19_10-12-14.tar.gz
[2026-06-19 10:12:15] Backup successful: ubuntu-server-backup-2026-06-19_10-12-14.tar.gz (2.3M)
[2026-06-19 10:12:15] Retention cleanup complete
ubuntu_server@ubuntu-server:~$ _
```

Create restore script

sudo nano /usr/local/bin/[restore.sh](#)

```
ubuntu_server@ubuntu-server: ~
GNU nano 6.2 /usr/local/bin/restore.sh *
#!/bin/bash
LOG_FILE="/backups/logs/restore.log"
BACKUP_DIR="/backups/daily"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

if [ -z "$1" ]; then
    echo "Usage: sudo $0 <backup-filename.tar.gz>"
    echo "Available backups:"
    ls -lh "$BACKUP_DIR"
    exit 1
fi

ARCHIVE="$BACKUP_DIR/$1"

if [ ! -f "$ARCHIVE" ]; then
    log "ERROR: Archive not found: $ARCHIVE"
    exit 1
fi

log "Starting restore from: $1"
tar -xzf "$ARCHIVE" -C /

if [ $? -eq 0 ]; then
    log "Restore successful from: $1"
else
    log "ERROR: Restore failed"
    exit 1
fi

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/ Go To Line  ^M-E Redo

Type here to search  3:15 AM 6/19/2026
```

Make it executable

sudo chmod 700 /usr/local/bin/restore.sh

Scheduled via cron (runs daily at 2:00 AM):

sudo crontab -e

Add this at the bottom

```
0 2 * * * /usr/local/bin/backup.sh >> /backups/logs/cron.log 2>&1
```

```
ubuntu_server@ubuntu: ~  
GNU nano 6.2 /tmp/crontab.QhJnB1/crontab *  
# Edit this file to introduce tasks to be run by cron.  
#  
# Each task to run has to be defined through a single line  
# indicating with different fields when the task will be run  
# and what command to run for the task  
#  
# To define the time you can provide concrete values for  
# minute (m), hour (h), day of month (dom), month (mon),  
# and day of week (dow) or use '*' in these fields (for 'any').  
#  
# Notice that tasks will be started based on the cron's system  
# daemon's notion of time and timezones.  
#  
# Output of the crontab jobs (including errors) is sent through  
# email to the user the crontab file belongs to (unless redirected).  
#  
# For example, you can run a backup of all your user accounts  
# at 5 a.m every week with:  
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/  
#  
# For more information see the manual pages of crontab(5) and cron(8)  
#  
# m h dom mon dow   command  
0 2 * * * /usr/local/bin/backup.sh >> /backups/logs/cron.log 2>&1_
```

Help Write Out Where Is Cut Execute Location M-U Undo
Exit Read File Replace Paste Justify Go To Line M-E Redo

Type here to search 3:20 AM 6/19/2026

8. Disaster Recovery Test

Phase 1: Confirm Working State (Before Disaster)

```
ubuntu_server@ubuntu: ~  
ubuntu_server@ubuntu:~$ curl http://192.168.10.20  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to nginx!</title>  
<style>  
  body {  
    width: 35em;  
    margin: 0 auto;  
    font-family: Tahoma, Verdana, Arial, sans-serif;  
  }  
</style>  
</head>  
<body>  
<h1>Welcome to nginx!</h1>  
<p>If you see this page, the nginx web server is successfully installed and  
working. Further configuration is required.</p>  
  
<p>For online documentation and support please refer to  
<a href="http://nginx.org/">nginx.org</a>.<br/>  
Commercial support is available at  
<a href="http://nginx.com/">nginx.com</a>.</p>  
  
<p><em>Thank you for using nginx.</em></p>  
</body>  
</html>  
ubuntu_server@ubuntu:~$
```

Phase 2: Simulate the Disaster

```
sudo rm -rf /var/www/html/*
```

```
sudo rm -f /etc/nginx/sites-enabled/default
```

```
ubuntu_server@ubuntu:~$ sudo rm -rf /var/www/html/*  
  
ubuntu_server@ubuntu:~$ sudo rm -f /etc/nginx/sites-enabled/default  
ubuntu_server@ubuntu:~$ curl http://192.168.10.20  
<html>  
<head><title>403 Forbidden</title></head>  
<body>  
<center><h1>403 Forbidden</h1></center>  
<hr><center>nginx/1.18.0 (Ubuntu)</center>  
</body>  
</html>  
ubuntu_server@ubuntu:~$
```

Phase 3: Restore From Backup

Check available backups

```
ls -lh /backups/daily/
```

```
ubuntu_server@ubuntu:~$ sudo ls -lh /backups/daily/  
total 2.3M  
-rw-r--r-- 1 root root 2.3M Jun 19 10:12 ubuntu-server-backup-2026-06-19_10-12-14.tar.gz  
ubuntu_server@ubuntu:~$
```

Run backup

sudo /usr/local/bin/restore.sh ubuntu-server-backup-2026-06-19_10-12-14.tar.gz

```
ubuntu_server@ubuntu-server:~$ sudo /usr/local/bin/restore.sh ubuntu-server-backup-2026-06-19_10-12-14.tar.gz

[2026-06-19 10:34:04] Starting restore from: ubuntu-server-backup-2026-06-19_10-12-14.tar.gz
[2026-06-19 10:34:05] Restore successful from: ubuntu-server-backup-2026-06-19_10-12-14.tar.gz
[2026-06-19 10:34:05] Restarting services...
[2026-06-19 10:34:05] Restore complete
ubuntu_server@ubuntu-server:~$
```

Phase 4: Verify Recovery

```
ubuntu_server@ubuntu-server:~$ curl http://192.168.10.20
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu_server@ubuntu-server:~$
```

```
ubuntu_server@ubuntu-server:~$ sudo nginx -t
[sudo] password for ubuntu_server:
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu_server@ubuntu-server:~$
```

```
ubuntu_server@ubuntu-server:~$ sudo ufw status
Status: active
```

To	Action	From
--	-----	----
2222/tcp	ALLOW	192.168.10.0/24
80/tcp	ALLOW	192.168.10.0/24
443/tcp	ALLOW	192.168.10.0/24

```
ubuntu_server@ubuntu-server:~$
```

◆ CIS Hardening Checklist

Control	Applied	Verified
Unnecessary packages removed	✓	✓
Unused services disabled	✓	✓
SSH password auth disabled	✓	✓
SSH on non-standard port	✓	✓
Root login disabled	✓	✓
UFW default-deny active	✓	✓
Source-restricted firewall rules	✓	✓
Fail2ban brute force protection	✓	✓
IP forwarding disabled	✓	✓
SYN cookies enabled	✓	✓
ICMP redirects disabled	✓	✓
Reverse path filtering enabled	✓	✓
Password minimum length (14)	✓	✓
Account lockout (5 attempts)	✓	✓
Password expiry (90 days)	✓	✓
/etc/shadow permissions (640)	✓	✓
Login banner configured	✓	✓
Automatic security updates	✓	✓

◆ Key Takeaways & Enterprise Security Impact

- **Minimal Install Philosophy:** Starting with a no-GUI server install forces CLI discipline and reflects real-world production practice. Every unnecessary package is a potential attack surface eliminated from day one.
- **Defense in Depth:** No single control is relied upon. SSH key auth eliminates brute force. UFW restricts by source and port. Fail2ban catches anomalies. Kernel hardening prevents network attacks. CIS controls reduce OS-level risk. Each layer independently reduces exposure.
- **Verified Recovery:** A backup never tested is an assumption, not a guarantee. The full disaster simulation proves the backup is usable, the restore process is documented, and recovery time (~8 minutes RTO) is measurable — skills directly applicable to real incident response scenarios.
- **Automation Over Manual:** Both the backup and restore processes are scripted and logged — eliminating human error during high-stress recovery scenarios and mirroring enterprise runbook practices.